

Lecture 3 : Regularization and Optimization

정규화(regularization)와 최적화(optimization)

= 딥러닝과 머신러닝 전반에서 매우 중요한 개념, 특히 CV에서 중요

지난 강의 복습

이미지 분류 = 이미지를 입력받아 정해진 라벨 집합(고양이, 개, 새, 사슴, 트럭 등) 중 하나로 매핑하는 것



분류가 어려운 이유

= 의미론적 차이 : 우리는 고양이를 보지만 컴퓨터에게는 픽셀 값 격자(다차원 배열, 텐서)일 뿐

= 조명 : 조명 위치에 따라 픽셀 강도가 달라지고 그림자에 가려진 부분은 보기 어렵다

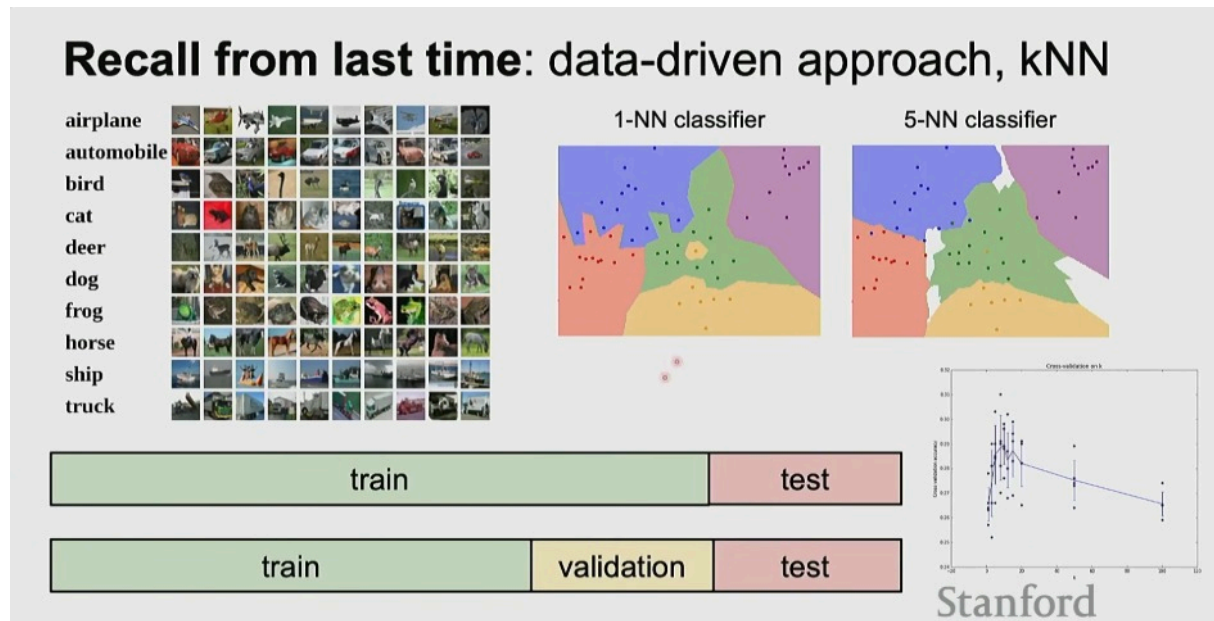
= 변형 : 고양이는 몸을 비틀고 구부려서 항상 같은 모양이 아니다

= 가려짐 : 쿠션 밑에 숨어 있어도 사람은 꼬리를 보고 고양이인 걸 알지만 컴퓨터는 어렵다

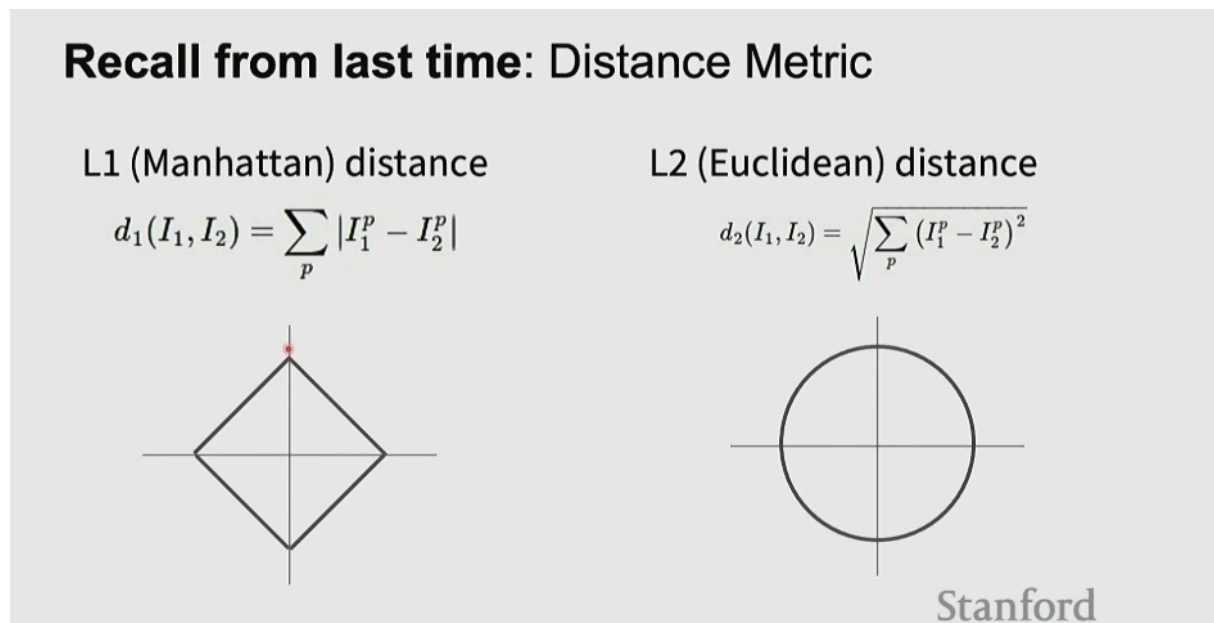
= 배경 혼란 : 물체가 배경과 섞여 보일 수 있다

= 클래스 내 변이 : 같은 범주 안에서도 생김새가 매우 다양하다

= if-else 규칙으로 해결할 수 없다 → 데이터 기반 접근



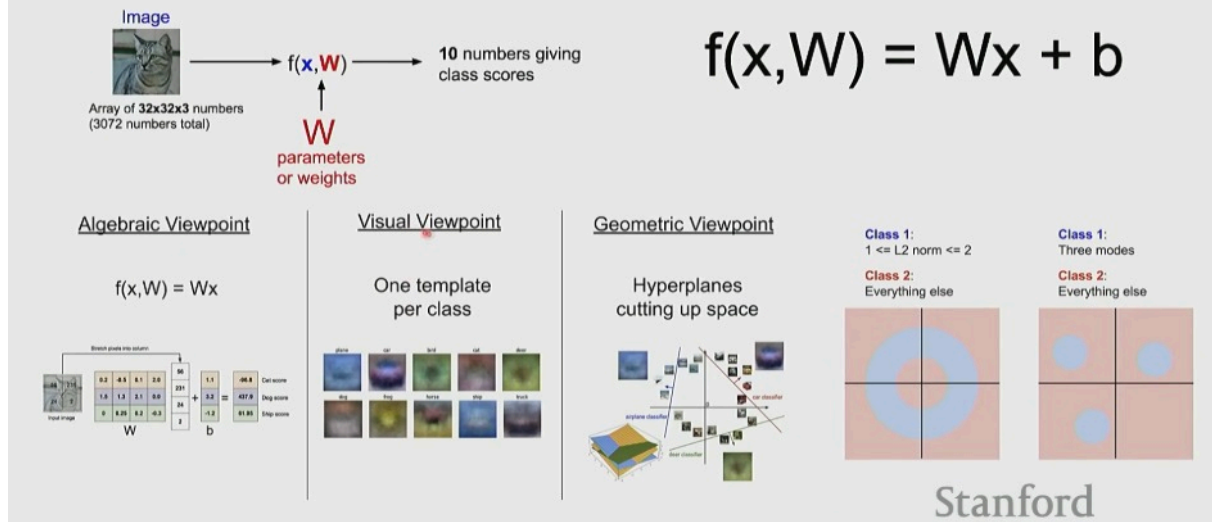
K 최근접 이웃 : 새 데이터와 거리가 가까운 훈련 데이터를 찾아 가장 많은 라벨을 할당
train / validation / test로 나누고 validation 셋으로 하이퍼파라미터(K) 선택, test 셋은 처음 보는 데이터에서의 성능 확인용



L2(유클리드) 거리 = 직선 거리. 원점에서 같은 거리의 점들이 원을 이룬다

L1(맨해튼) 거리 = 대각선으로 못 가고 상하좌우로만 이동. 위로 0.5 + 오른쪽으로 0.5 = 1
이라서 마름모 위의 점들이 원점에서 같은 거리

Recall from last time: Linear Classifier



선형 분류기 : $32 * 32 * 3$ 이미지를 3072개 숫자로 펴서 $10 * 3072$ 가중치 행렬 W 를 곱하고 bias를 더하면 10개 클래스 점수

세 가지 관점

1. 대수적 관점 : W 의 각 행이 독립적으로 한 클래스를 담당, 입력 x 와 곱하고 bias를 더해 점수 계산
2. 시각적 관점 : W 의 각 행을 이미지 모양으로 되돌리면 클래스별 템플릿
3. 기하학적 관점 : W 의 각 행은 입력 공간의 직선(식 = 0인 결정 경계). 선 위쪽은 양수 점수, 아래쪽은 음수 점수

= 기하학적 관점의 장점 : 데이터를 시각화하면 직선 하나로 완벽히 나눌 수 없다는 걸 바로 알 수 있다

Q. 템플릿은 KNN의 이웃 하나와 수학적으로 같은가?

A. 아니다. 템플릿은 특정 데이터 하나가 아니라 선(결정 경계)으로부터 만들어진 것

손실 함수

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

Where x_i is image and y_i is (integer) label

Loss over the dataset is a average of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Stanford

고양이는 car 점수가 더 높아서 틀렸고, 자동차는 맞췄고, 개구리는 정답 점수가 가장 낮아서 완전히 틀렸다

= 직관적으로 이 점수들이 안 좋다는 건 알지만, 이걸 수학적으로 어떻게 나타낼까? = 손실 함수

= 분류기가 얼마나 나쁜지 알려준다

= 각 훈련 예제 x_i 를 모델 $f(x_i, W)$ 에 넣어 나온 결과를 정답 y_i 와 비교해서 손실 계산 → 전체 데이터에 대해 평균

= 분류에서 가장 많이 쓰는 건 softmax 손실(교차 엔트로피 손실). 정답 클래스를 낮은 확률로 예측하면 손실이 크고, 높은 확률로 예측하면 손실이 작다

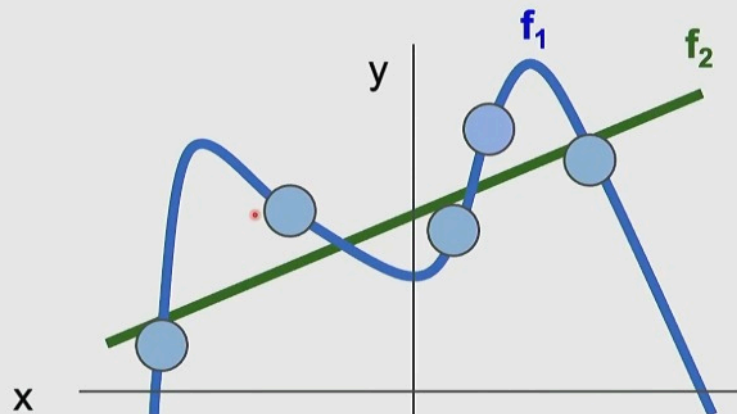
정규화

데이터 손실(data loss) = 모델 예측이 훈련 데이터와 얼마나 잘 맞는지. 낮을수록 훈련 데이터를 잘 맞춘 것

정규화 항(regularization) = 모델이 훈련 데이터에 너무 잘 맞추는 것을 막는다

= 훈련 데이터에서는 오히려 성능이 떨어지지만, 새로운 테스트 데이터(처음 보는 데이터)에서는 더 잘하게 만드는 것이 목표

Regularization intuition: Prefer Simpler Models



Stanford

정규화의 직관 : 더 단순한 모델을 선호

= f1은 모든 점을 지나서 데이터 손실이 매우 낮다

= f2는 모든 점을 완벽하게 지나지는 않지만, 새로운 데이터로 테스트하면 f2가 더 좋은 모델일 것 같다

= 데이터에 과적합하지 말고, 덜 맞추더라도 더 단순한 모델이 나올 수 있다

= 오컴의 면도날 : 여러 가설이 경쟁할 때는 가장 단순한 것부터 선택하고, 그게 확실히 틀리면 더 복잡한 것을 시도한다

Regularization

λ = regularization strength
(hyperparameter)

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too well* on training data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

More complex:

Dropout

Batch normalization

Stochastic depth, fractional pooling, etc

Stanford

λ (람다) = 정규화 강도. 또 다른 하이퍼파라미터라서 train, validation 셋으로 최적값을 정한다

= 0이면 정규화 없음, 커질수록 점점 더 강한 정규화 = 모델이 훈련 데이터에 얼마나 맞추지 못하게 할지 조절하는 손잡이

간단한 예시

1. L2 정규화 : 가중치 행렬의 각 원소를 제곱해서 모두 더한 값 $\times \lambda$ 를 손실에 더함
2. L1 정규화 : 제곱 대신 절댓값

L2 vs L1의 차이

= L2는 제곱하기 때문에 0.001 같은 작은 값은 제곱하면 더 작아진다 = 작은 값에 대한 벌점이 매우 작다 $\rightarrow$ 작지만 0은 아닌 값들이 고르게 퍼진 가중치를 선호

= L1은 제곱하지 않아서 작은 값도 그대로 벌점 $\rightarrow$ 0이거나 0에 매우 가까운 값이 많아진다
= 희소(sparse)한 가중치

= 어떤 값이 0이어도 성능이 비슷하다면 L1은 그 값을 0으로 밀어낸다

= 최적화는 데이터 손실과 정규화 항의 합을 최소화하는 것이라서, 데이터 손실이 크게 변하지 않으면서 정규화 항을 낮출 수 있으면 그쪽을 선호하게 된다

더 복잡한 정규화 : 드롭아웃, 배치 정규화 등 = 모델의 층 자체를 바꾸기도 한다. 매년 새 논문이 나오는 연구 분야

정규화를 하는 이유

1. 가중치에 대한 선호를 표현할 수 있다 (고르게 퍼진 가중치 = L2, 희소한 가중치 = L1)
2. 모델을 더 단순하게 만들어서 테스트 데이터에서 더 잘 작동하게 한다 (예 : 고차 다항식 항에 강한 정규화)
3. 최적화를 개선할 수 있다. 특히 L2는 제곱(포물선) 형태라 볼록(convex)해서 전역 최솟값이 있는 좋은 성질 $\rightarrow$ 모델을 더 빨리 학습시키기도 한다 (이 강의 범위 밖)

Regularization: Expressing Preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regularizer prefer?

$$w_1^T x = w_2^T x = 1$$



Stanford

Q. $x = [1, 1, 1, 1]$ 일 때 $w_1 = [1, 0, 0, 0]$ 과 $w_2 = [0.25, 0.25, 0.25, 0.25]$ 는 x 와 내적하면 둘 다 점수 1로 데이터 손실이 같다. L2 정규화는 어느 쪽을 선호할까?

A. w_2 . 0.25를 제공하면 1/16, 4개 더하면 1/4. w_1 은 1. 정규화 손실이 4배 낮다 = 고르게 퍼진 가중치 선호

Q. L1 정규화는 어느 쪽을 선호할까?

A. 함정 문제. 둘 다 절댓값의 합이 1이라서 L1 손실은 같다. (실제로는 희소성 때문에 w_1 쪽이 나오는 경우가 많다)

Q. 정규화가 어떻게 모델을 단순하게 만드나?

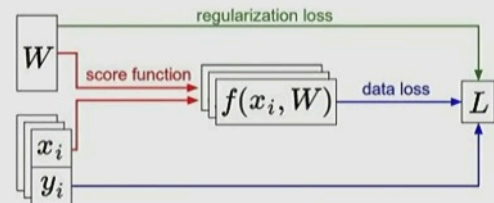
A. L1은 0이 많은 가중치 = 계수가 적은 선형 모델이라 단순하다. 하지만 정규화가 항상 단순한 모델을 만드는 건 아니다. 핵심은 훈련 데이터에서 덜 잘하고 테스트 데이터에서 더 잘하는 것. 드롭아웃처럼 오히려 모델을 복잡하게 만드는 정규화도 있다

Recap

- We have some dataset of (x, y)
- We have a **score function**: $s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$
- We have a **loss function**:

$$L_i = -\log\left(\frac{e^{s y_i}}{\sum_j e^{s_j}}\right) \text{ Softmax}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W) \text{ Full loss}$$



Stanford

정리

= (x, y) 쌍의 데이터셋과 점수 함수(선형 모델 = 행렬 곱)가 있다

= softmax 손실 : 점수에 지수를 취해 모두 양수로 만들고 전체 합으로 나눠 확률 분포로 만든 뒤, 정답 라벨 확률의 $-\log$

= 전체 손실 = 모든 훈련 예제의 L_i 평균 + 가중치에 대한 정규화 항

Q. softmax를 쓰는 이유?

A. 어떤 실수 목록이든 합이 1인 확률 분포로 바꿔준다. 큰 양수는 거의 1, 매우 작은 음수는 거의 0

최적화

= 주어진 W 가 얼마나 좋은지 계산하는 방법은 알았다. 그러면 가장 좋은 W 는 어떻게 찾을 까?

손실 지형(loss landscape) : 높이(z 축)가 손실, $x \cdot y$ 축이 모델의 파라미터 2개인 지형

= 그 위를 걸어다니면서 가장 낮은 지점을 찾는 것

= 사람은 멀리 보면 계곡이 어디인지 보이지만, 실제로는 눈을 가린 사람처럼 지금 서 있는 곳의 경사만 느낄 수 있다고 생각하면 매우 정확한 비유

Strategy #1: A first very bad idea solution: **Random search**

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
    W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
    loss = L(X_train, Y_train, W) # get the loss over the entire training set
    if loss < bestloss: # keep track of the best solution
        bestloss = loss
        bestW = W
    print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (truncated: continues for 1000 lines)
```

Stanford

전략 1 : 무작위 탐색 (매우 나쁜 방법)

= for문으로 W를 1000번 무작위로 뽑아서 가장 좋은 것 선택

= CIFAR-10에서 정확도 15.5%. 무작위보다는 낫지만 좋지 않다 (최신 딥러닝은 99.7%)

Strategy #2: **Follow the slope**



Stanford

전략 2 : 경사를 따라 내려가기

= 눈을 가린 채 발밑의 땅을 느끼면서 경사가 가리키는 아래쪽으로 계속 걸어가는 것

= 이 수업의 모든 모델, 사실상 모든 딥러닝 모델이 학습되는 기본 방식

Strategy #2: Follow the slope

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension

The slope in any direction is the **dot product** of the direction with the gradient
The direction of steepest descent is the **negative gradient**

Stanford

1차원 : 도함수. 현재 위치에 아주 작은 h를 더한 곳의 함수값에서 현재 함수값을 빼고 h로 나눈 뒤 $h \rightarrow 0$ 극한

다차원 : 기울기(gradient) = 각 차원별 편미분을 모은 벡터

= 어떤 방향으로의 경사 = 그 방향과 gradient의 내적

= gradient는 언덕 위쪽을 가리키고, 가장 가파르게 내려가는 방향은 음의 gradient(negative gradient)

current W:	W + h (first dim):	gradient dW:
[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,...]	[0.34 + 0.0001, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,...]	[-2.5, ?, ?, ?, ?, ?, ?, ?, ?, ?,...]
loss 1.25347	loss 1.25322	$(1.25322 - 1.25347)/0.0001 = -2.5$ $\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$

Stanford

수치적 gradient : W의 한 값에 0.0001을 더해서 손실이 얼마나 바뀌는지 보고, 그 차이를 h로 나눠 근사

= W의 모든 값에 대해 이 과정을 반복

= 문제점 : 모든 값을 하나씩 돌아야 해서 매우 느리고, 근사값이라 정확하지 않으며 부동소수점 연산 오차도 크다

This is silly. The loss is just a function of W:

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

$$L_i = -\log\left(\frac{e^{s y_i}}{\sum_j e^{s_j}}\right)$$

$$s = f(x; W) = Wx$$

want $\nabla_W L$

Stanford

그런데 손실은 그냥 W에 대한 함수다

= 손실은 W, x_i , y_i 의 함수. x_i 와 y_i 는 상수로 두고 W에 대해서만 미분($\nabla_W L$)

= 미적분의 연쇄 법칙 등으로 계산 가능 = 해석적 gradient (과제에서 직접 유도)

In summary:

- Numerical gradient: approximate, slow, easy to write
- Analytic gradient: exact, fast, error-prone

=>

In practice: Always use analytic gradient, but check implementation with numerical gradient. This is called a **gradient check**.

Stanford

정리

1. 수치적 gradient : 근사값, 느림, 하지만 작성하기 쉽다

2. 해석적 gradient : 정확, 빠름, 하지만 직접 새로 짜면 오류가 생기기 쉽다

= 실제로는 해석적 gradient를 쓰고, 수치적 gradient로 구현이 맞는지 확인 = gradient check (과제에서도 함)

Q. 미분이 불가능한 더 좋은 손실 함수가 있다면 수치적 방법으로 쓸 수 있나?

A. 가능할 수도 있지만, 미분 불가능한 더 좋은 손실 함수를 만들기는 어렵다. 손실이 대부분의 영역에서 미분 불가능하고 서로 연결되지 않은 점들처럼 생겼다면 가장 가파른 방향으로 내려가도 좋은 해에 도달하지 못한다

= 함수가 볼록(convex)하면 경사 하강 방식이 매우 잘 작동하지만, 미분 불가능하고 볼록하지 않으면 잘 안 된다

경사 하강법 (Gradient Descent)

Gradient Descent

```
# Vanilla Gradient Descent

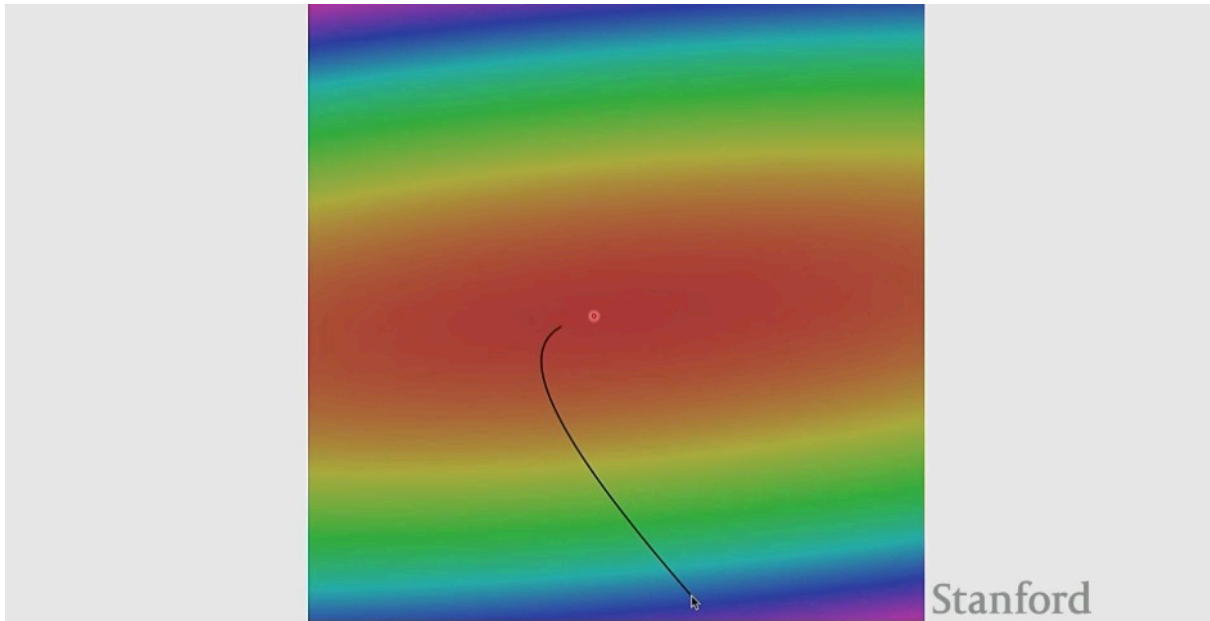
while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

Stanford

= 손실 지형의 각 지점에서 경사를 계산하고 아래쪽으로 한 걸음 이동

= 손실 함수, 데이터, 현재 가중치로 gradient를 계산 → 각 가중치를 얼마나 바꿔야 내려가는지 알 수 있다

= 보폭(step size, learning rate)만큼 음의 gradient 방향으로 이동 : $\text{weights} += -\text{step_size} \times \text{weights_grad}$



위에서 내려다본 손실 지형. 보라색이 가장 높은 곳, 빨간색이 계곡 바닥

= 보폭은 고정이지만 바닥 근처에서는 평평해져서 gradient가 작아지기 때문에 실제 이동 거리도 작아진다

Q. 언제 멈추나?

A. 위 코드는 무한 반복. 보통 정해진 반복 횟수만큼 돌리거나, 손실이 거의 줄지 않으면($1e-5$, $1e-9$ 정도) 충분하다고 보고 멈춘다

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Full sum expensive
when N is large!

Approximate sum
using a **minibatch** of
examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent
```

```
while True:
```

```
    data_batch = sample_training_data(data, 256) # sample 256 examples
```

```
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
```

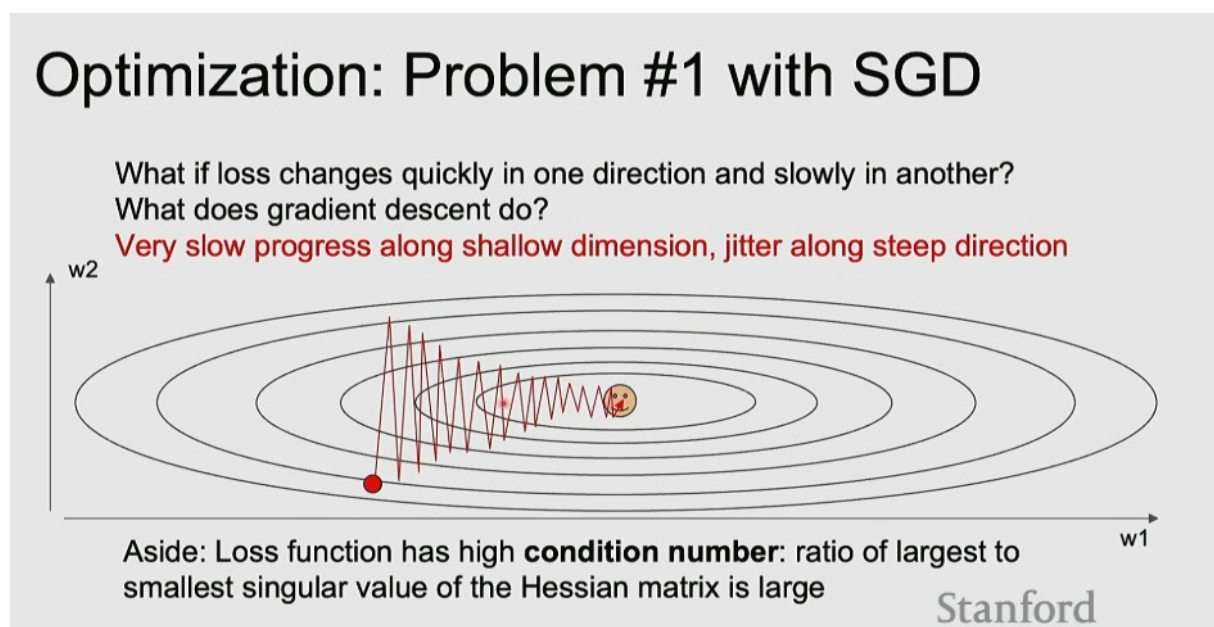
```
    weights += - step_size * weights_grad # perform parameter update
```

Stanford

확률적 경사 하강법 (SGD, Stochastic Gradient Descent)

- = 경사 하강법은 전체 훈련 데이터 N개의 손실을 다 합해서 계산 → 데이터가 크면 계산량이 너무 많다
- = SGD는 매번 데이터의 일부(미니배치)만 보고 계산. 예) 256개 샘플링 → 그 256개로 gradient 계산 → 업데이트
- = 매 단계마다 무작위로 일부를 뽑아서 stochastic(확률적)
- = 실제로는 완전히 무작위로 뽑지 않고, 데이터를 무작위 순서로 한 번씩 다 돈 뒤 다시 반복
- = 1 epoch
- = 미니배치 크기는 32 / 64 / 128이 흔하다

SGD의 문제점



문제 1 : 한 방향으로는 가파르고 다른 방향으로는 완만한 좁은 계곡

등고선(level set) = 손실 값이 같은 점들을 이은 선

= 가파른 방향으로 위아래로 크게 흔들리고, 완만한 방향으로 매우 느리게 진행 = 지그재그(jitter)

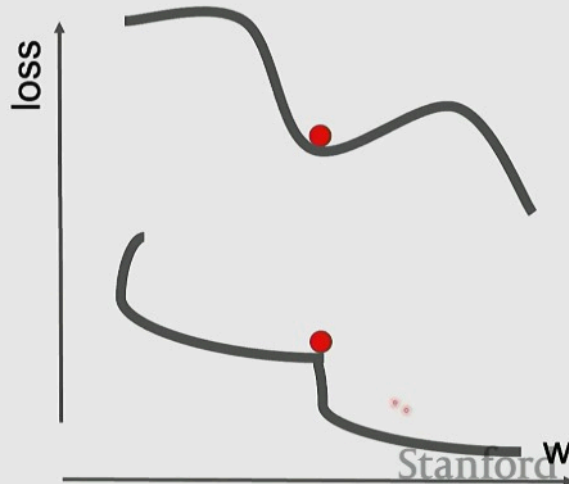
= 보폭이 크고 경사가 충분히 가파르면 계곡 밖으로 튕겨나갈 수도 있다 (learning rate가 너무 클 때 실제로 발생)

= 수학적으로는 조건수(condition number, 헤시안 행렬의 최대/최소 특이값 비율)가 큰 경우. 위아래 방향 2차 도함수는 매우 크고 좌우 방향은 매우 작다

Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

Zero gradient,
gradient descent
gets stuck



문제 2 : 지역 최솟값(local minima)이나 안장점(saddle point)

= gradient가 0이라서 경사 하강법이 멈춰버린다

= 조금만 더 가면 훨씬 더 내려갈 수 있는데도 갇힌다

= 차원이 높아질수록(파라미터가 많을수록) 안장점이 훨씬 더 흔하다

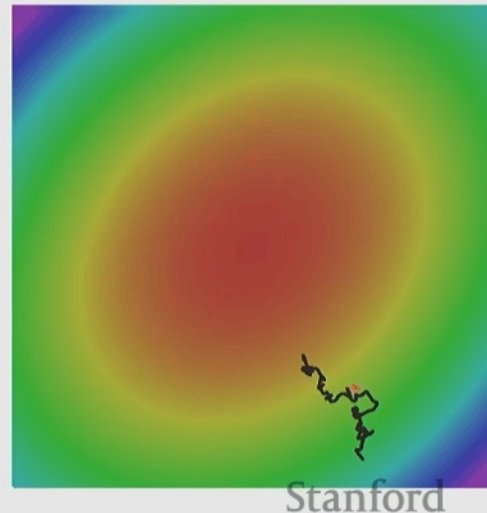
= 안장점 : 말안장처럼 생겨서 중심에서 모든 방향의 gradient가 0

Optimization: Problem #3 with SGD

Our gradients come from minibatches so they can be noisy!

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W)$$

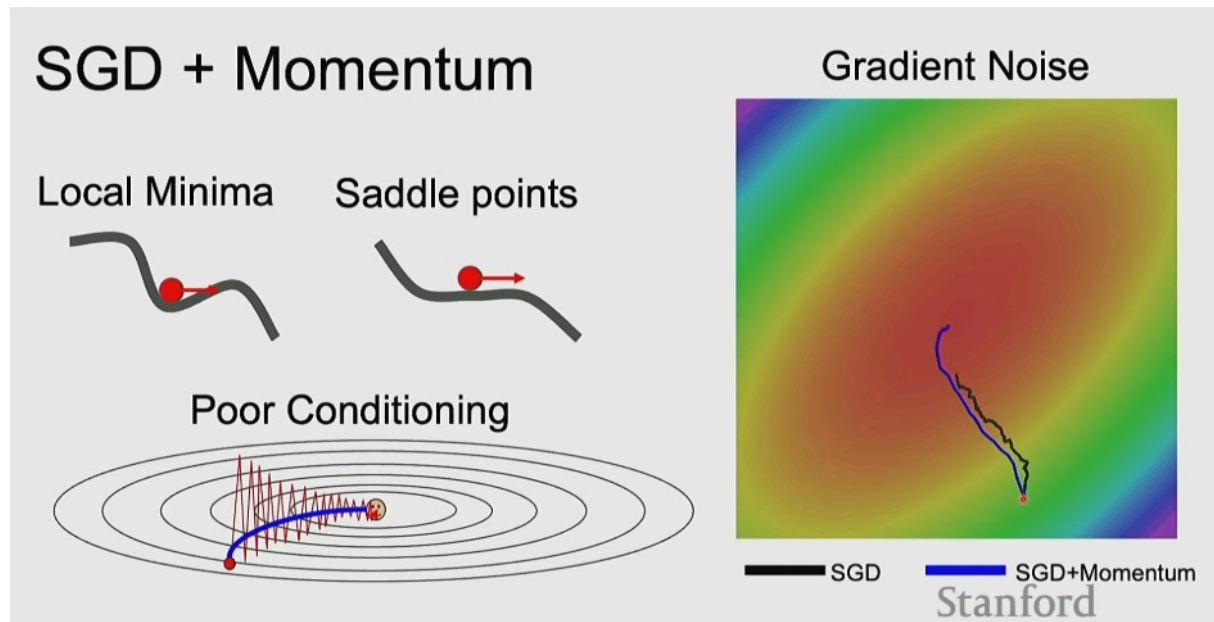
$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W)$$



문제 3 : 미니배치에서 gradient를 계산하기 때문에 노이즈가 있다

= 전체 데이터가 아니라 일부만 보기 때문에 각 단계가 최솟값을 향해 곧바로 가지 않고 흔들리면서 진행

SGD + Momentum



= 언덕을 굴러 내려가면서 속도가 붙는 공을 생각하면 된다. 실제 물리 성질과도 비슷하게 모델링

= 지역 최솟값 : 충분한 속도로 굴러오면 빠져나올 수 있다

= 안장점, 평평한 곳 : 계속 굴러왔기 때문에 멈추지 않고 계속 간다

= 나쁜 조건수 : 위아래 진동은 여전히 있지만, 오른쪽으로 계속 가는 방향으로 속도가 쌓여서 중심으로 더 빨리 간다

= gradient 노이즈 : 여러 gradient가 공통으로 향하는 방향(최솟값 쪽)이 momentum에 반영되어 노이즈가 평균화된다

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

- Build up “velocity” as a running mean of gradients
- Rho gives “momentum”; typically rho=0.9 or 0.99

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

Stanford

SGD : $x -= \text{learning_rate} \times dx$

SGD + Momentum : $vx = \text{rho} \times vx + dx \rightarrow x -= \text{learning_rate} \times vx$

= gradient 대신 속도(velocity)로 업데이트

= 속도 = 이전 속도 $\times$ rho + 현재 gradient = 최근 gradient들의 이동 평균

= rho(momentum) : 과거와 현재를 얼마나 반영할지. 보통 0.9나 0.99. 클수록 이전 속도에 더 의존

= 진동하는 방향은 서로 반대 방향이라 상쇄되어 덜 움직인다

= 같은 식을 다른 형태로 쓰기도 하지만 결과는 같다

Q. 정확히 안장점 한가운데를 따라 굴러가면?

A. 가능은 하지만 거의 일어나지 않는다. 그래서 실제로는 다른 random seed로 여러 번 학습하고, SGD는 노이즈가 있어서 빠져나오는 경우가 많다

Q. 안장점은 SGD만의 문제인가?

A. 전체 데이터로 계산해도 생기고, 오히려 더 흔할 수 있다. 부가 기법 없이 gradient descent만 쓰는 최적화 알고리즘 모두의 문제

Q. momentum 때문에 지나쳤다가 돌아오느라 수렴이 어려워지지 않나?

A. 그럴 수 있다. 수렴은 더 느릴 수 있지만 평균적으로 더 좋은 최솟값을 찾는다. 결국 경험적인 것이라 여러 개를 시도해서 가장 잘 되는 걸 쓴다

RMSProp

More Complex Optimizers: RMSProp

SGD +
Momentum

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

Adds element-wise scaling of the gradient based on the historical sum of squares in each dimension (with decay)

RMSProp

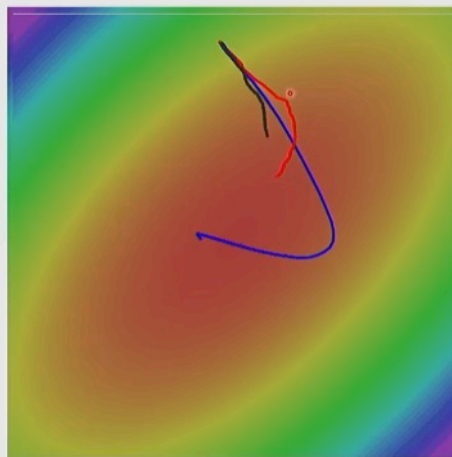
```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Tieleman and Hinton, 2012

Stanford

- = 2012년 제프리 힌턴 그룹. momentum처럼 속도를 쌓는 대신 gradient를 스케일링
- = $\text{grad_squared} = \text{decay_rate} \times \text{grad_squared} + (1 - \text{decay_rate}) \times dx \times dx =$ gradient 제곱의 이동 평균
- = 업데이트할 때 $\text{learning_rate} \times dx$ 를 $\sqrt{\text{grad_squared}}$ 로 나눈다 (원소별)
- = gradient가 큰(가파른) 방향은 큰 값으로 나뉘서 덜 이동, gradient가 작은(평평한) 방향은 작은 값으로 나뉘서 더 이동
- = 좁은 계곡에서 평평한 방향으로 더 많이 움직이게 해 준다

RMSProp



- SGD
- SGD+Momentum
- RMSProp

Stanford

세 가지 비교

= SGD + Momentum : 지나쳤다가 다시 돌아온다

= SGD : 고정된 방향으로만 움직여서 느리다

= RMSProp : 가파른 방향은 gradient 제공 항이 커서 덜 움직이고, 빠르게 평평한 중심 쪽으로 방향을 튼다

Adam

= 현대 딥러닝에서 가장 많이 쓰는 최적화 기법. SGD + Momentum과 RMSProp을 합친 것

Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```

Momentum

RMSProp

Sort of like RMSProp with momentum

Q: What happens at first timestep?

Kingma and Ba, "Adam: A method for stochastic optimization", ICLR 2015

Stanford

= first_moment (빨간 부분) = momentum. β_1 이 momentum 계수, gradient의 이동 평균 = 속도

= second_moment (파란 부분) = RMSProp의 gradient 제곱 이동 평균

= 업데이트 : $\text{learning_rate} \times \text{first_moment} / \sqrt{\text{second_moment}}$

= 평평한 방향으로 가속, 가파른 방향으로 감속 + 같은 방향으로 계속 가면 속도가 붙는다

Q. 첫 번째 time step에서는 무슨 일이 생길까?

A. β_1, β_2 는 보통 0.9, 0.999처럼 1에 가깝고 first, second moment는 0으로 초기화

= 첫 단계에서 second_moment가 0에 매우 가깝다 $\rightarrow$ 0에 가까운 값으로 나누게 되어 gradient가 작았는데도 첫 걸음이 매우 커진다 = 원하지 않는 동작

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with **beta1 = 0.9**, **beta2 = 0.999**, and **learning_rate = 1e-3 or 5e-4** is a great starting point for many models!

Kingma and Ba, "Adam: A method for stochastic optimization", ICLR 2015

Stanford

Adam (full form)

= 편향 보정(bias correction) 항을 추가 = moment 추정값이 0에서 시작한다는 점을 보정. time step t에 따라 달라진다

= t가 커질수록 보정 항은 거의 필요 없어진다 (과제에서 직접 구현)

= 좋은 기본값 : beta1 = 0.9, beta2 = 0.999, learning_rate = 1e-3 또는 5e-4 = 많은 모델의 좋은 출발점

= Adam은 RMSProp과 SGD + Momentum의 특징을 둘 다 가지고 수렴한다

AdamW: Adam Variant with Weight Decay

Q: How does regularization interact with the optimizer? (e.g., L2)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Computed after the moments!

AdamW (Weight Decay) adds term here

Stanford

AdamW : Weight Decay를 적용한 Adam 변형

Q. 정규화(예 : L2)는 최적화 기법과 어떻게 상호작용할까?

= 기본 Adam은 gradient를 계산할 때 L2 항까지 포함 → 데이터 손실 + 정규화 손실 둘 다로 moment 계산

= AdamW는 데이터 손실만으로 moment를 계산하고, 정규화 항은 마지막에 따로 더한다 (moment 계산 이후)

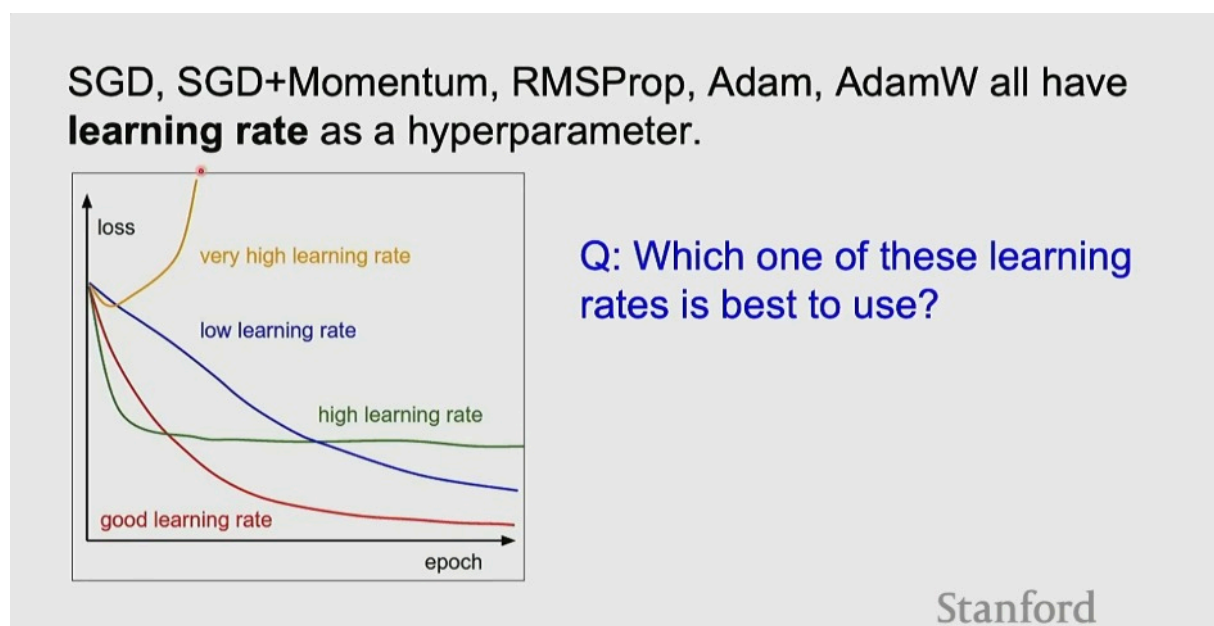
= weight decay = moment 계산에 넣지 않고 마지막에 L2 정규화를 더하는 것

= 많은 설정에서 AdamW가 조금 더 잘 된다. 메타의 Llama 시리즈도 AdamW 사용

Q. 왜 두 개로 나누나?

A. 속도와 momentum이 가중치 값이 아니라 손실의 함수가 되게 하려고. 가중치 값과 독립적으로 손실 지형을 이동하고 싶지만 정규화 항은 필요할 때 moment 계산을 방해하지 않도록. 결국 경험적이라 둘 다 해 보고 더 좋은 걸 쓴다

학습률 (Learning Rate)



SGD, SGD + Momentum, RMSProp, Adam, AdamW 모두 learning rate가 하이퍼파라미터

= 매우 높은 학습률 : 손실 지형 밖으로 튕겨나가면서 손실이 폭발

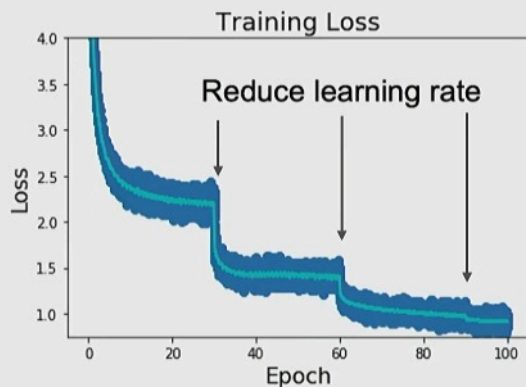
= 낮은 학습률 : 수렴이 매우 느리다

= 높은 학습률 : 튕겨나가지는 않지만 최솟값 주변에서 부딪히기만 하고 더 내려가지 못한다

= 좋은 학습률 : 손실이 빠르게 줄고 학습을 계속할수록 꾸준히 개선

= 실제로는 상황과 학습 단계에 따라 여러 개가 다 좋은 학습률일 수 있다

Learning rate decays over time



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Stanford

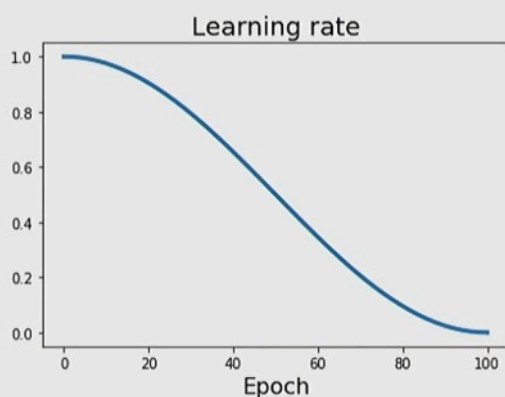
학습률은 학습 중에 바꿀 수 있다 = 최신 좋은 모델들은 거의 다 학습 중에 학습률을 바꾼다

Step decay : 정해진 반복 횟수마다 학습률을 1/10로 줄인다

= 학습률이 너무 높아서 더 수렴하지 못할 때 줄이면 손실 지형에서 더 내려갈 수 있다

= ResNet 학습에 많이 쓴다 (예 : 30, 60, 90 epoch에서 0.1배). 그래프에서 줄이는 지점이 계단처럼 보인다

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2} \alpha_0 (1 + \cos(t\pi/T))$$

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
 Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
 Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
 Child et al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

α_0 : Initial learning rate
 α_t : Learning rate at epoch t
 T : Total number of epochs

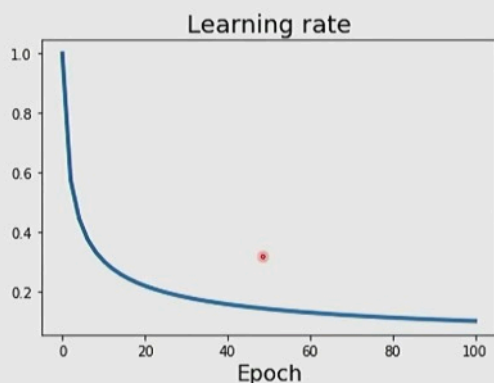
Stanford

Cosine 학습률 감소 : 최대 학습률에서 시작해서 코사인 반주기 모양으로 끝에 0까지 내려간다. 매우 많이 씀

= 학습 중반에 꾸준한 개선이 보이는 손실 모양

= 어떤 스케줄러를 쓰느냐에 따라 학습 중 손실 곡선 모양이 완전히 달라진다

Learning Rate Decay



Vaswani et al, "Attention is all you need", NIPS 2017

Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0 (1 - t/T)$

Inverse sqrt: $\alpha_t = \alpha_0 / \sqrt{t}$

α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

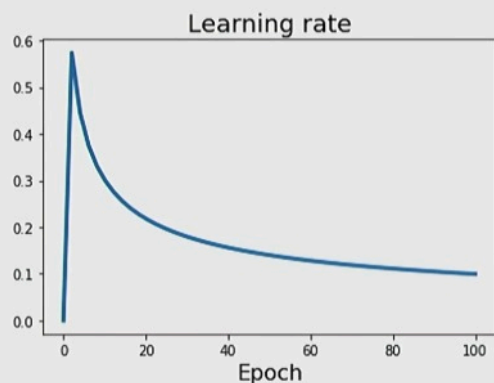
Stanford

Linear 감소 : 직선으로 줄인다

Inverse sqrt : $1/\sqrt{t}$ 로 줄인다

= 학습률을 바꾸는 방법은 무한히 많다. 모델 종류에 따라 가장 잘 되는 걸 고른다

Learning Rate Decay: Linear Warmup



Goyal et al, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour", arXiv 2017

High initial learning rates can make loss explode; linearly increasing learning rate from 0 over the first ~5,000 iterations can prevent this.

Empirical rule of thumb: If you increase the batch size by N , also scale the initial learning rate by N

Stanford

Linear Warmup

= 처음부터 최대 학습률로 시작하면 손실이 폭발할 수 있다 → 처음 약 5,000번 반복 동안 0에서 최대값까지 선형으로 올린 다음 원래 스케줄러 사용

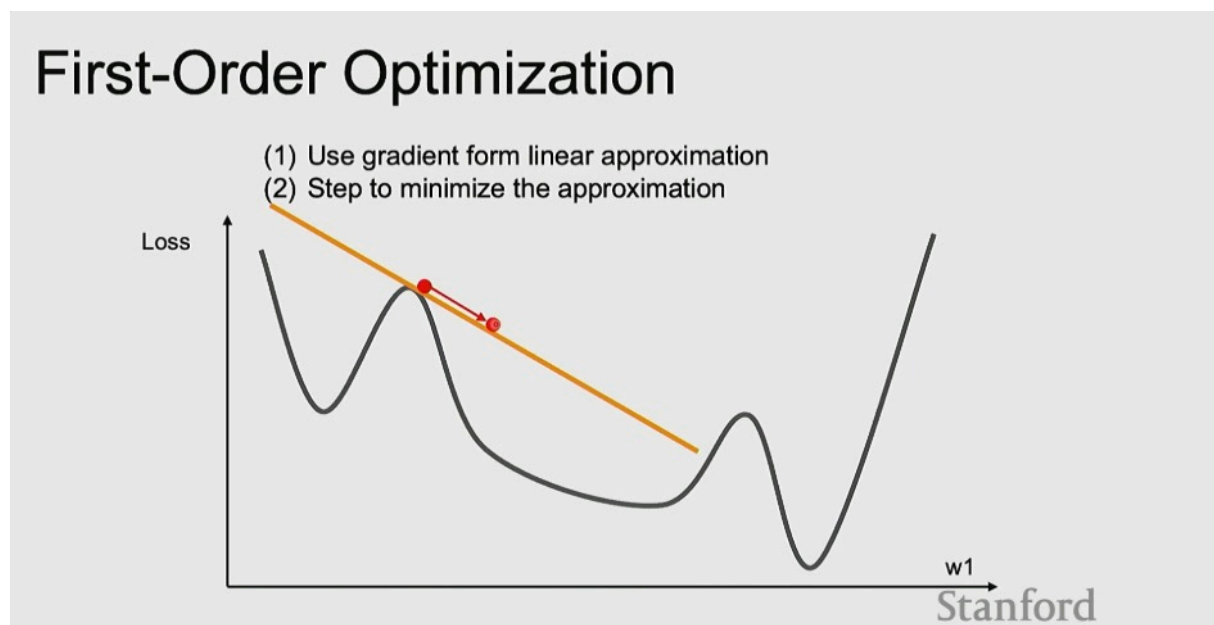
= warmup + inverse sqrt, warmup + cosine이 인기 있는 조합

선형 스케일링 법칙 (경험 법칙)

= 배치 크기를 N배 늘리면 초기 학습률도 N배 늘린다

= 수학적 증명 시도도 있지만 주로 많은 문제에서 경험적으로 확인된 것. 잘 되는 설정에서 배치 크기를 늘리고 싶으면 학습률도 같은 비율로 늘리기

2차 최적화 (Second-Order Optimization)



1차 최적화 : gradient로 선형 근사를 만들어 아래쪽 방향을 찾고 그 방향으로 한 걸음 이동.
momentum, RMSProp 등을 더해도 기본은 gradient

Second-Order Optimization

second-order Taylor expansion:

$$J(\theta) \approx J(\theta_0) + (\theta - \theta_0)^\top \nabla_{\theta} J(\theta_0) + \frac{1}{2} (\theta - \theta_0)^\top H(\theta - \theta_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$\theta^* = \theta_0 - H^{-1} \nabla_{\theta} J(\theta_0)$$

Q: Why is this bad for deep learning?

Stanford

2차 최적화 : gradient 대신 헤시안(2차 도함수)을 이용해서 그 지점에 2차 다항식을 맞추고 그 최솟값으로 바로 이동 (테일러 전개, 뉴턴 방법)

= 일부 최적화 문제에서는 매우 잘 작동

Q. 왜 딥러닝에서는 안 쓸까?

A. 1. 2차 혼합 도함수를 계산해야 해서 어렵다

1. 모든 파라미터 × 모든 파라미터 크기의 행렬이라, 파라미터가 수백만~수십억 개인 신경망에서는 너무 커서 GPU 메모리가 부족하다

= 작은 모델이거나 시간을 더 써서 더 좋은 걸음을 원하면 고려할 만하지만, 큰 신경망에서는 헤시안 계산에 시간을 쓰느니 데이터를 더 보는 게 낫다

실제로는

In practice:

- **Adam(W)** is a good default choice in many cases; it often works ok even with constant learning rate
- **SGD+Momentum** can outperform Adam but may require more tuning of LR and schedule
- If you can afford to do full batch updates then look beyond 1st order optimization (**2nd order and beyond**)

Stanford

= Adam(W)는 새로운 문제에서 첫 모델을 학습할 때 좋은 기본 선택. 학습률을 고정해도 꽤 잘 된다

= Adam(W) + 고정 학습률, 또는 Adam(W) + linear warmup + cosine decay가 인기 있는 조합

= SGD + Momentum이 Adam보다 좋을 때도 있지만 학습률과 스케줄을 훨씬 더 많이 튜닝해야 한다 (RMSProp 향이 없어서)

= Adam은 여러 분야에서 검증되었고 손실 지형에 잘 적응한다

= 한 번에 전체 훈련 데이터를 배치로 넣을 수 있다면(full batch) 데이터나 모델이 크지 않은 것이니 2차 이상의 최적화를 고려해 볼 만하다